COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/21/26	**DATE SUBMITTED:**	10/21/26
NAME:	Cruz, Elijah Villaruel, Prince Benjamin	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14 **Algorithm Complexity**

1. PROCEDURE OUTPUT

Outputs (Table 14-1)

Algorithm 1

Output

```
Algorithm 1
n case insertion merge
100 best 99 356
100 worst 4950 316
1000 best 999 5044
1000 worst 499500 4932
10000 best 9999 69008
10000 worst 49995000 64608
```

Algorithm 2

Output

```
Algorithm 2
n case insertion(us) merge(us)
100 best 0.301703 3.05216
100 worst 7.80246 3.06547
1000 best 2.7377 40.2309
1000 worst 780.092 39.8144
10000 best 21.882 482.566
10000 worst 74426.1 621.364
```

	Best Case	Worst Case	Analysis
Algorithm 1	**O(n)**	**O(n²)**	The average case is O(n²) and space is O(1) because it sorts in place. It is very fast on sorted or nearly sorted data and slow on large reversed data.
Algorithm 2	**O(n log n)**	**O(n log n)**	Input order barely matters, since it always splits and merges. Space is O(n) for the temporary array here.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Algorithm 1	Algorithm 2
n = 100, Best case	0.42	3.28
n = 100, Worst case	9.72	3.29
n = 1,000, Best case	2.46	50.18
n = 1,000, Worst case	780.59	53.22
n = 10,000, Best case	22.66	643.18
n = 10,000, Worst case	93,833.4 (about 94 ms)	820.92

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

Analysis
As to what we have observed with the output, insertion sort was quick on sorted data but was slow on the reversed data. Making the list 10x bigger made the worst case slower, and at n = 10,000 it was faster than merge sort on sorted data. Merge sort took about the same time for sorted and reversed data, and making the list 10× bigger made it only about 20x slower. That made it about faster than insertion sort on reversed data at n = 10,000. The results match the theory: insertion sort depends on how the data is arranged (O(n) at best, O(n²) at worst), while merge sort does not (O(n log n) either way).

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode

Linear Search

```
START
FUNCTION LinearSearch(array, target)
        FOR EACH element IN array
                IF element == target THEN
                        RETURN index
                END IF
        END FOR
        RETURN -1
END FUNCTION
END
```

Binary Search

```
START
FUNCTION BinarySearch(array, target)
        SET low = 0
        SET high = length(array) - 1

        WHILE low <= high
                SET mid = (low + high) / 2
                IF array[mid] == target THEN
                        RETURN mid
                ELSE IF array[mid] < target THEN
                        SET low = mid + 1
                ELSE
                        SET high = mid -1
                END IF
        END WHILE

        RETURN -1
END FUNCTION
END
```

Time and Space Complexities

Algorithm	Best Time	Average TIme	Worst Time	Space Complexity
Linear Search	O(1)	O(N)	O(N)	O(1)
Binary Search	O(1)	O(log N)	O(log N)	O(1)

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	33.298	1343.4
10,000	49.017	1343.4
100,000	61.615	137010

Analysis
In our empirical testing, linear search runtime grew linearly with input size N, increasing tenfold as N scaled up (1.343 to 137,010), confirming its O(N) complexity.Conversely, binary search exhibited O(log N) growth, scaling minimally from 33.3 to 61.6. We can see that binary search is vastly superior for large datasets, provided the array is pre-sorted.

Conclusion
Villaruel I learned that algorithm speed comes down to input size and big-O complexity, not just hardware. Benchmarking showed how fast algorithms get bogged down with big datasets. For the sorting procedure, insertion sort was faster on tiny or already sorted inputs, but merge sort dominated as N grew. In the supplementary activity, binary search (O(log N)) stayed super fast while linear search (O(N)) slowed way down. I think I handled the code and analysis pretty well, since it is based on our previous laboratory activities, but I can still work on writing cleaner code next time. **Cruz** This activity showed me that how fast an algorithm runs depends mostly on the size of its input and its big-O complexity, and not only on the computer it runs on. Benchmarking made it clear how quickly algorithms slow down once the data gets large. In the sorting part, insertion sort did better on small or already sorted inputs, while merge sort pulled far ahead as n grew. In the supplementary activity, binary search (O(log N)) stayed very fast, while linear search (O(N)) got much slower as N increased. I think I did well on the code and analysis because they built on our earlier laboratory activities, but I still want to work on writing cleaner code in the future.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **GEMINI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *grammar correction, code assistance, explanation of concepts*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: *grammar correction, code assistance, explanation of concepts*

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.